IC405/EC307 Algorithm

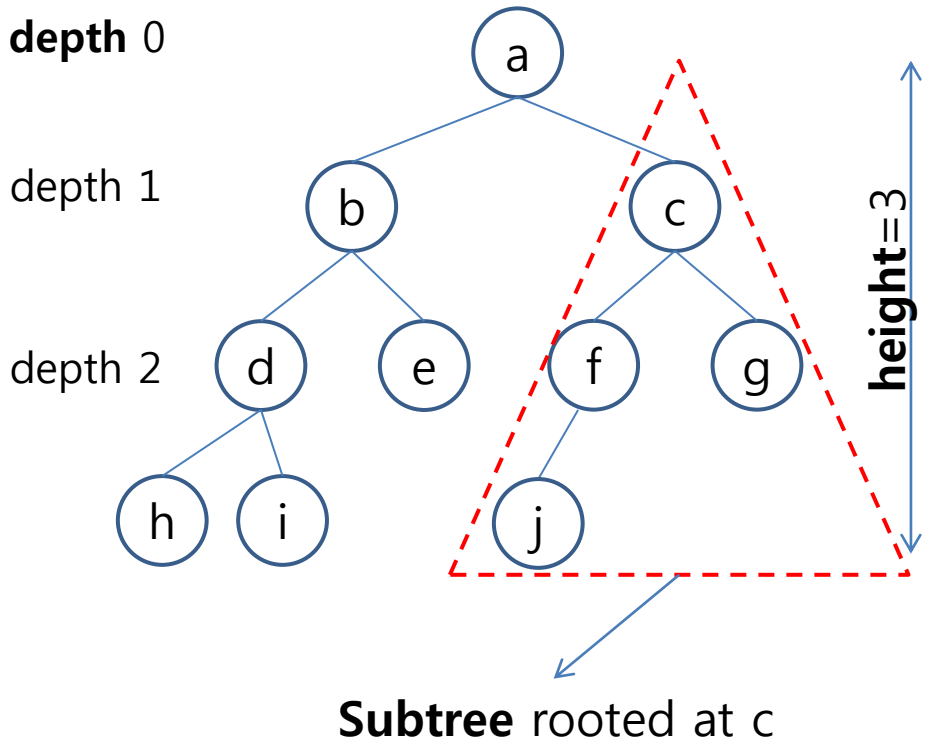
Heapsort and Linear Time Sorting Algorithms

GIST
Sung-Hee Lee

Announcement

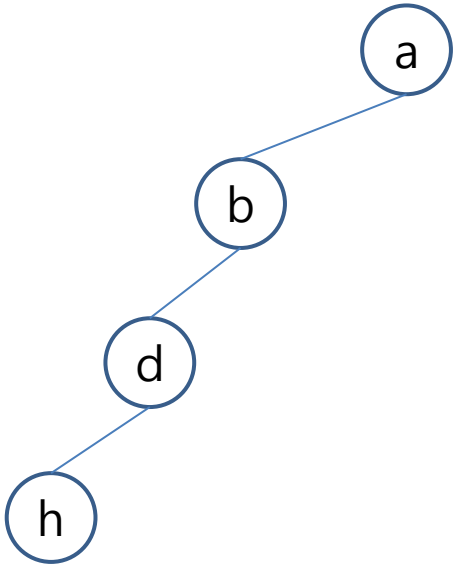
- Quiz scheduled for 3/29(Thur)
- Visit the class web page for the problem set
  - Don’t need to submit the solution
  - Quiz will be based on the problem set

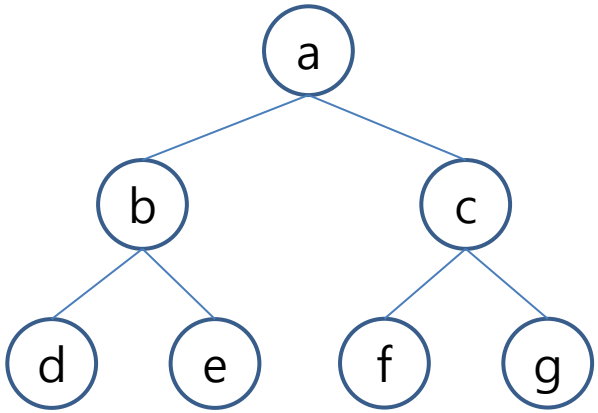
(Binary) Tree Data Structure



- **Root**: a
- **Leaf nodes**: h,i,e,j,g
- **Internal nodes**: non-leaf nodes
- b is d's **parent**, d is b's **child**
- b is h's **ancestor**, h is b's **descendant**
- **Siblings** have the same parent: {d,e}, {f,g}, {h,i}…

Examples of Binary Trees



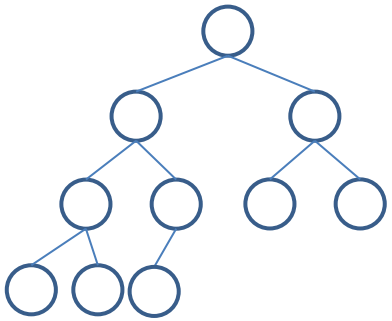


complete tree

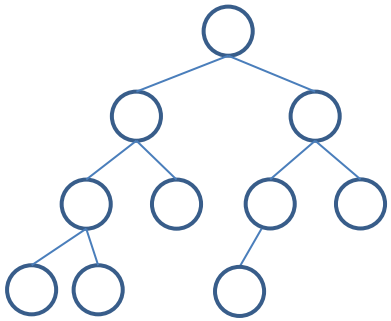
- n: # of nodes
- $\mathrm{floor}(\lg n) \leq$ height $\leq$ n-1

Heap Data Structure

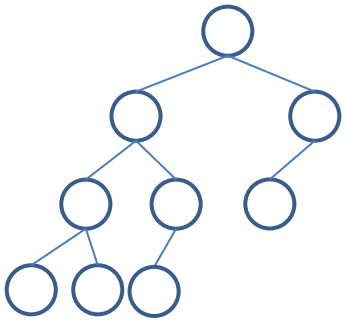
- Heap data structure
  - A tree-based data structure with the “heap property”
- Balanced, left-justified binary tree
  - Balanced tree: “complete tree” except the lowest level
  - Left-justified: leaves at the lowest level are filled from the left



balanced, left-justified



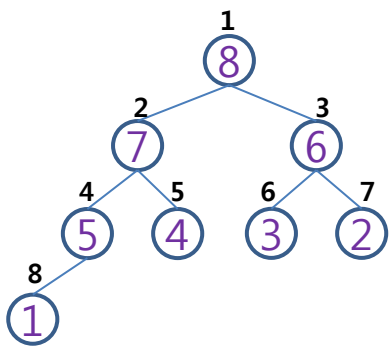
balanced, not left-justified

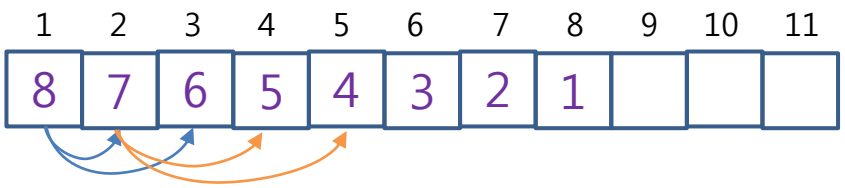


not balanced

Heap Data Structure

- For n nodes, height h= $\lfloor \lg n \rfloor$ ($\lfloor\ \rfloor$: floor)
- For h height, the number of nodes is
  $2^h \leq n \leq 2^{h+1} - 1$
- Balanced, left-justified binary tree can be represented with an array





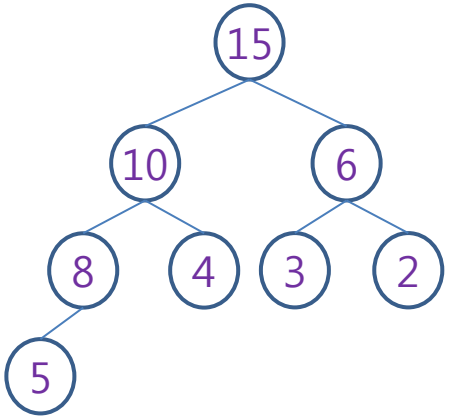
Left_child(i)=2i

Right_child(i)=2i+1

Parent(i)= $\lfloor i/2 \rfloor$

Max-Heap

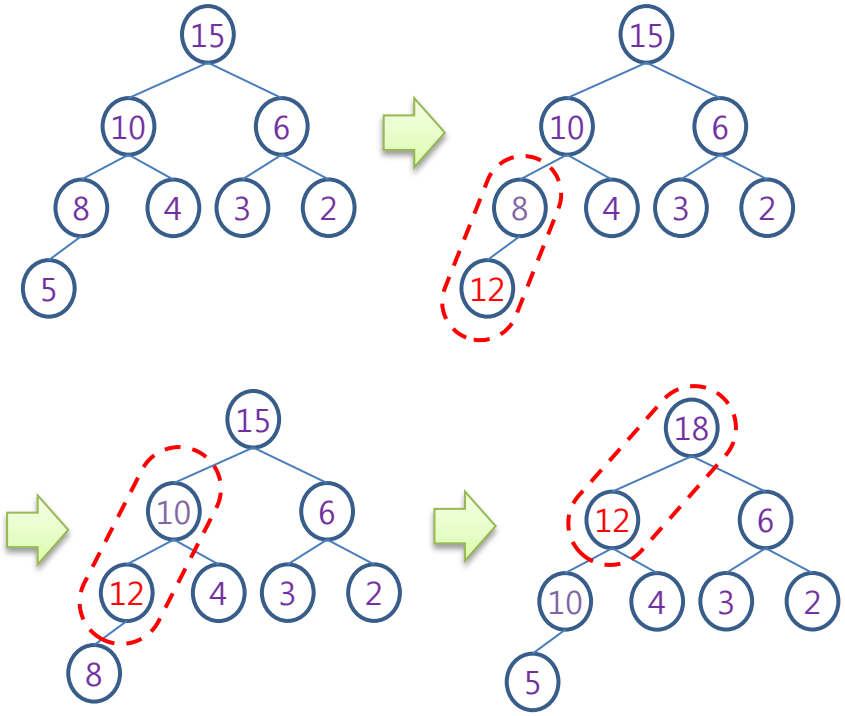
- Max-heap property:
  parent node's value ≥ child node's values
- Example



- A node has the maximum value of its sub-tree
  - but the value may be smaller than those of sibling's descendants (e.g., 6 and 8 above)
  - Max heap itself is not a sorted structure
- Likewise, you can define the min-heap

Maintaining Heap Property

How to maintain heap property if you increase a node's value

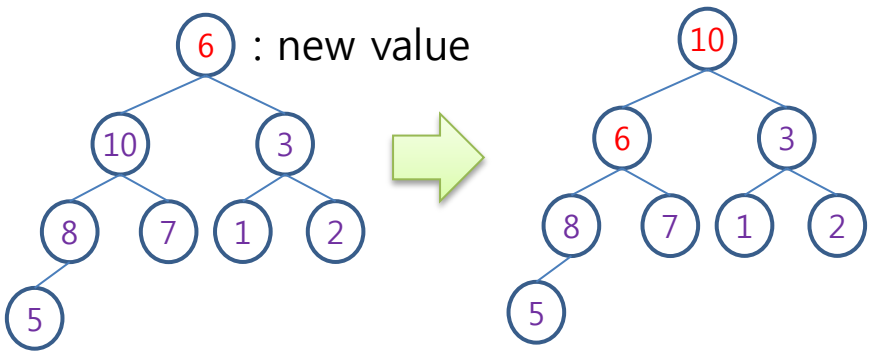


HEAP-INCREASE-KEY(A,i,key)

```
//increase A[i] to key
    if A[i]>key, return error
    A[i]=key
    while i>1 and A[parent(i)]<A[i]
        swap A[parent(i)] and A[i]
        i=parent(i)
```

Maintaining Heap Property

How to maintain heap property if you change the value of root



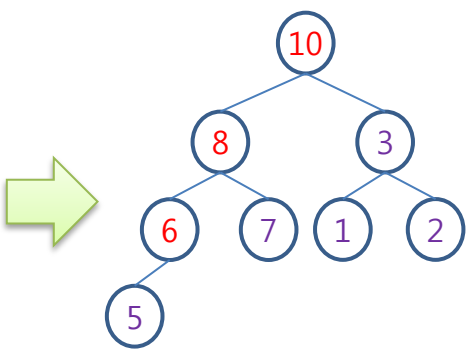
MAX-HEAPIFY(A,i)

j=index of i's child w/ larger value

if A[i]<A[j]

swap A[i] and A[j]

MAX-HEAPIFY(A,j)

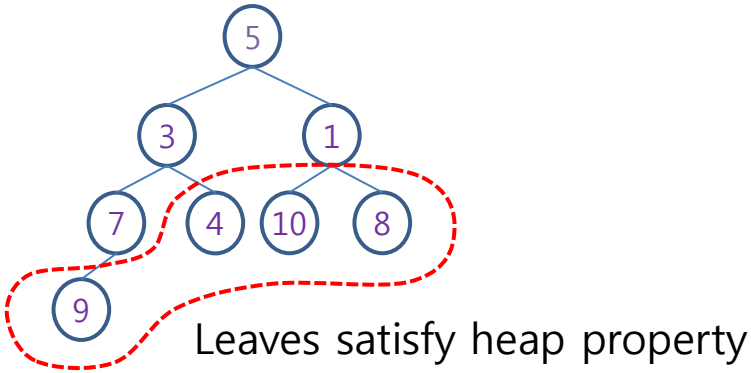


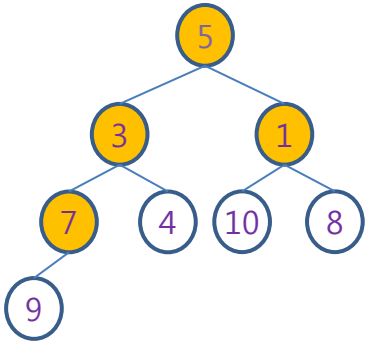
Max-heapify is O(h) (= O(lgn))

Compare with children and swap with a suitable child if needed

Building a Heap from a Random Array

- Idea





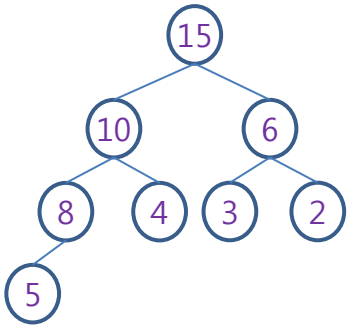
For internal nodes, from bottom to top, run MAX-HEAPIFY

```
BUILD-MAX-HEAP(A)
    n=A.length
    for i=floor(n/2) downto 1
        MAX-HEAPIFY(A,i)
```

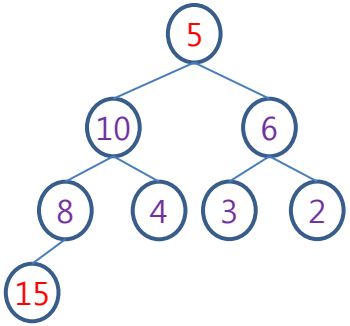
- Why from floor(n/2)?
  - Leaf nodes are A[floor(n/2)+1,…,n]
  - Verify this!
- Running time is O(nlgn)
  - More precisely, it is O(n) (Refer to textbook)
- Exercise
  - Build max heap from the left example

Heapsort

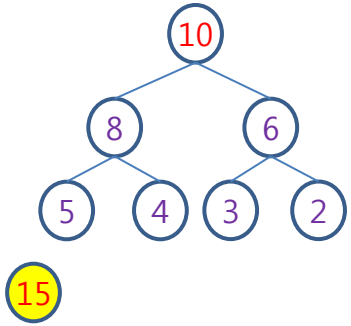
- Idea:



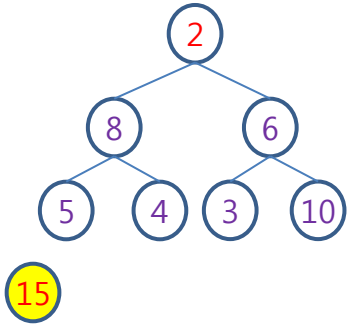
First, build max heap



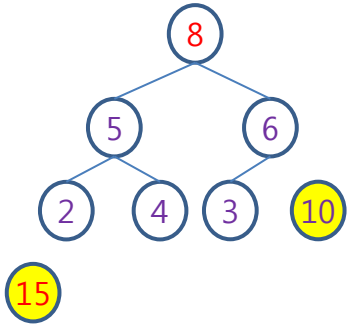
swap root and last nodes



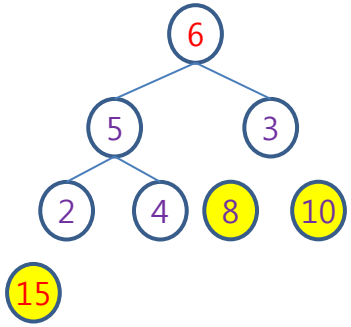
decrease heap length by 1 and max-heapify

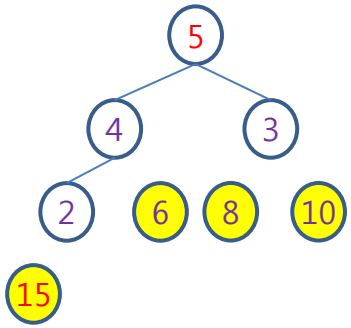


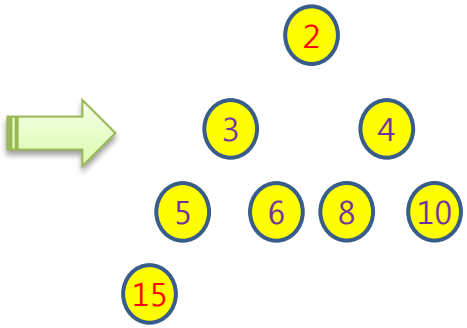
swap root and last nodes



decrease heap length by 1 and max-heapify







<u>Note that this operation is done inside an array</u>

Heapsort

```
HEAPSORT(A)
   BUILD-MAX-HEAP(A)
   for i=A.length downto 2
      swap A[1] and A[i]
      A.heap_size-=1
      MAX-HEAPIFY(A,1)
```

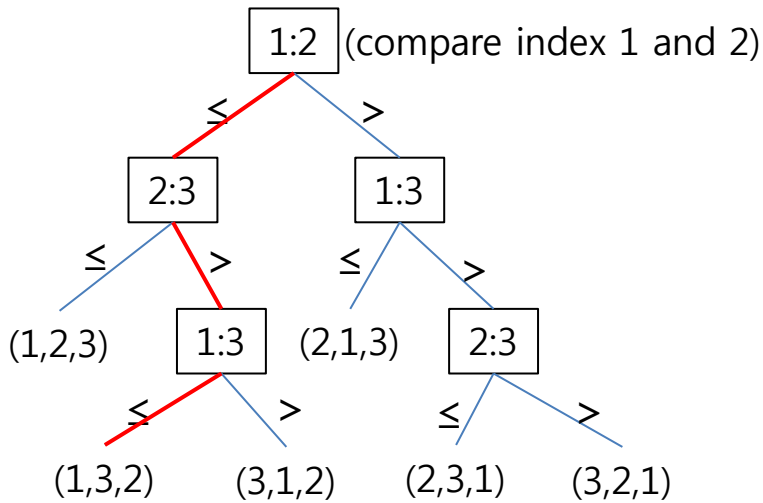
- Running time
  - Best case
    - Array is already a max heap
    - O(nlgn)
  - Worst case
    - O(nlgn)
- Heapsort is in-place sorting algorithm
  - No extra storage necessary

Comparison Sorts

- Until now, we have learned
  - Insertion sort
  - Merge sort
  - Quicksort
  - Heapsort
- All algorithms above achieve sorting only by comparing between input elements
  → called **comparison sort** algorithms
- Best worst-case running time for comparison sorts was $\Omega(n\lg n)$ (merge sort, heapsort).
  - Can we sort faster than $\Omega(n\lg n)$?

Decision-Tree Model

- Example
  - Sort 3 elements



  - e.g., sorting [4,6,5]
    (1,3,2)→[4,5,6]

- Decision-tree can model execution of any comparison sort
  - Binary tree
  - Node i:j → compare i^{th} & j^{th} elements
  - Leaf: indices of sorted array
  - After some number of comparisons, it reaches a leaf
  - Running time of the algorithm = length of the path taken
  - Worst case running time = height of tree

Lower Bound for Decision Tree Sorting

- Theorem: Any decision tree that can sort n elements must have height $\Omega(n\lg n)$
- Proof
  - Number of leaves L must be at least n!
    (n! = all permutations of the array)
  - L must be at most 2^h (from binary tree property)
  - $n! \leq L \leq 2^h$
  - $h \geq \lg(n!) = \Theta(n \lg n)$

    ← Stirling's approximation : $n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right)$
- Therefore, heapsort and merge sort are asymptotically optimal comparison sorts

Sorting in Linear Time

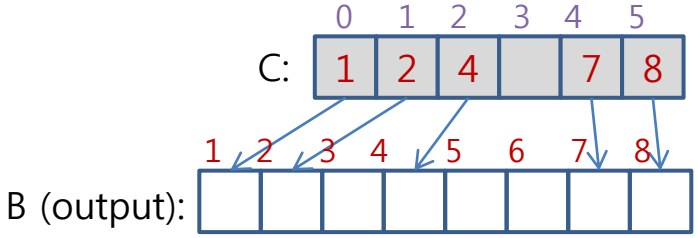
- There exist linear time sorting algorithms, which are not comparison sorts
- Examples
  - Counting sort
  - Radix sort
  - Bucket sort

Counting Sort

- Assumption
  - All n elements are integers in range [0,k]
  - Example: 0 2 4 5 2 4 1 4 → n=8, k=5

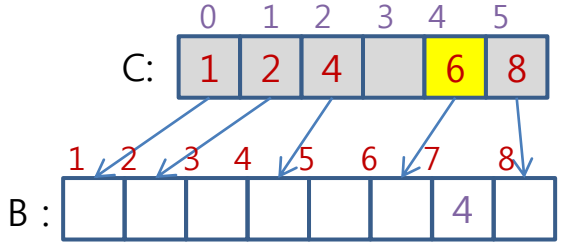
- Idea
  - Given input A:

  - Create an auxiliary array [0..k] indicating the index (of output array) to put each integer

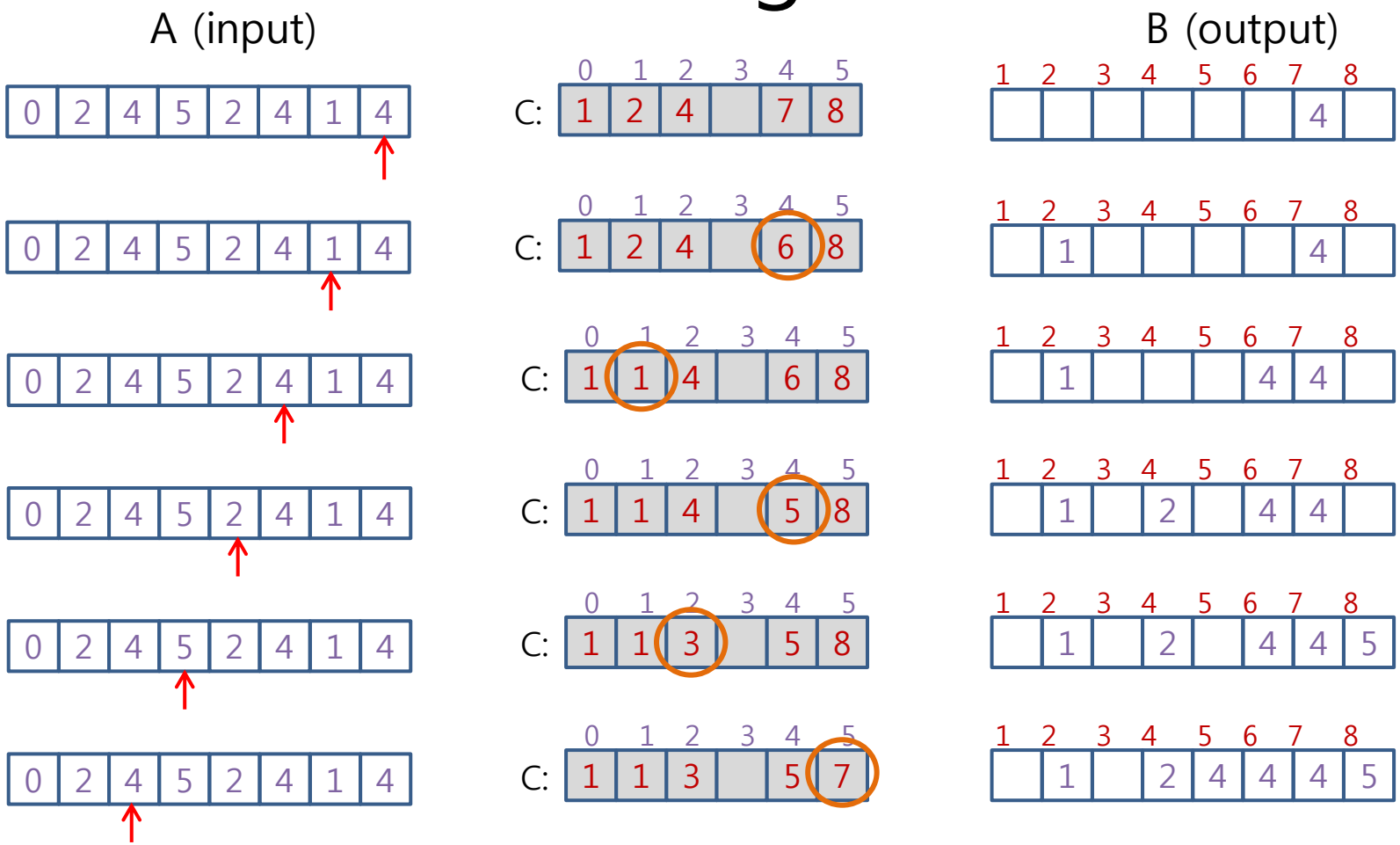


  - For each integer t in A, put t in B and decrease the index for the next appearance of same integer

After putting 4, C becomes



Counting Sort

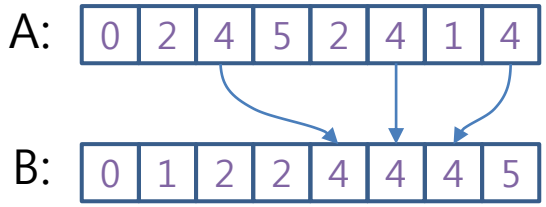


Input is sorted from right to left

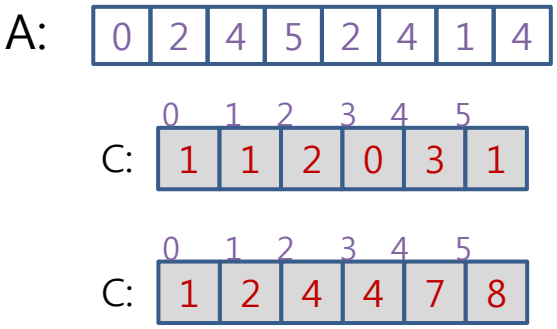
Running time: $\Theta(n)$

Counting Sort

- Counting sort is "stable" sort
  - Same integers keep their order in input and output arrays



- How to create the auxiliary array C[0..k]?
  1. Initially set C to 0's
  2. For i=1 to n, C[a]=C[a]+1 (a=A[i])
     (Now, C has counts of each integer)
  3. For i=0 to k, C[i]=C[i]+C[i-1]
     (Now, C has each integer's index in B)



  - Running time of creating C = Θ(n+k)

Counting Sort

- Linear time algorithm
  - Counting sort doesn't compare any elements
  - $\Theta(n+k)+\Theta(n)=\Theta(n+k)$

    creating C sorting
  - Counting sort is used when $k<<n$, then $\Theta(n)$

- Stable sort
  - Counting sort will be used for radix sort

- Limited to sorting integers of fixed range

Radix Sort

- Sort on digits from the least significant digit
- Example

272	350	401	257
453	401	350	272
401	272	453	350
350	453	257	401
257	257	272	453
587	587	587	587

sort on digit 1 → sort on digit 2 → sort on digit 3

```
//A: input array
//d: number of digits
RADIX-SORT(A,d)
    for i=1 to d
        stable-sort A on digit i
```

- Running time
  - $\Theta(d(n+k))$ if counting sort is used for stable sorting
  - For decimal number, k=10
  - When d is constant (which is usually the case), it is linear time algorithm
- What if we sort from the most significant digit?